

Sintaxa Prologului

Laboratorul 1 e mai mult despre sintaxa prologului.

Exemplu predicat (=functie):

```
culoare(rosu). % rosu e cu litera mica, deci constanta. Afirmam ca rosu e culoare.
culoare(albastru). % acum daca avem culoare(X), X va fi rosu apoi albastru.
factorial(0,1). % cazul de baza al recursiei. Factorial de 0 e 1.
factorial(N,F) :- N>0, PN is N-1, factorial(PN,G), F is G*N. % luam factorial de N-1 cu
rezultatul G, apoi il inmultim cu N si ne da rezultatul F.
```

- variabilele incep cu litera mare (sau underscore).
- constantele si predicatele (functii mai smechere) se scriu cu litera mica.
- “,” = și. “;” = sau. “\+” = not.
- noi doar definim predicate (functii) si constante. Pentru a face efectiv ceva, trebuie sa INTEROGAM (online e in dreapta-jos, in consola incarcam un fisier si dupa direct).
- INTEROGARE: ?- are(ana, X). -> ne va zice ce are ana (exemplu afisare: X = mere [tastati ; sau Next] X = pere)
- prologul e bazat pe UNIFICARE. asta e un fel de backtracking (pattern matching) pe toate solutiile posibile. UNIFICAREA este mai generala decat o functie, deoarece orice parametru poate fi de intrare sau iesire.
- predicatele nu returneaza ceva, ele sunt doar adevarate sau false. Trebuie sa folosim variabile ca sa avem un rezultat.
- listele functioneaza asa: primul element e HEAD, restul e TAIL. Se scrie [H|T]. cand nu mai are elemente se ajunge la [].
- setof(Variabile_la_care_vrem_valoarea, Conditiiile, Rezultat) ex: setof((X,Y), (member(X,[1,3]), member(Y, [2,4]), X < Y), Rezultat). -> Rezultat = [(1,2), (1,4), (3,4)]

Ciudatenii:

- "=" unifica (incarca sa "transforme" sa fie acelasi lucru si in stanga si in dreapta).
- "=:=" verifica egalitatea intre 2 constante (incearca sa le evalueze, deci variabilele trebuie sa fie numere).
- "!" opreste fortat executia mai departe (de pe acest branch)
- pt a calula $n + 1$, trebuie sa facem "N1 is N + 1", nu mergea cu = (ar fi fost unificat si ne facea N1 sa fie inlocuit cu stringul "N + 1"), trebuie cu "is".

Greseala comuna:

- predicatul "not\1" (\1 inseamna de aritate 1, adica are 1 parametru) e de preferat sa il folosim ca not((conditie1,conditie2)), a 2-a paranteza fiind necesara sa nu creada ca ii dam mai mult de 1 parametru.

*/

Laboratorul 2

Contine: operatii logice, demonstratii multimi

```
:- [lab2lmc1]. % ASA INCARCAM LABORATORUL ca sa mearga predicatele:
```

```
implica(P,Q) % definit ca :- not(P);Q.  
echiv(P,Q) % definit ca :- implica(P,Q), implica(Q,P).  
xor(P,Q) % definit ca :- P, not(Q) ; Q, not(P).
```

% 1. De ce merge cu tabele de adevar sa demonstram pt multimi?

(ddaca = daca si numai daca)
 pt orice multimi, $A = B$ ddaca (x apartine A ddaca x apartine B).
 x apartine A poate fi ori adevarat ori fals.
 Daca A este dat de o formula mai complexa: $A = C$ reunit cu D , atunci putem face o tabela de adevar: x apartine A daca x apartine C SAU x apartine D .
 Posibilitatile sunt $\{00, 01, 10, 11\}$ si doar pentru prima x nu e in A .

Daca avem in ambele parti formule mai interesante, putem doar verifica pentru o tabela de adevar sa dea mereu true-true sau false-false si atunci sunt egale.

Ex: (pt A,B multipli, $_a, _b$, P,Q reprezinta daca un element e sau nu in multime.
A inclus sau egal cu B => implica($_a, _b$)
A inclus (si diferit) in B => implica(P,Q), not(implica(Q,P)).
A reunit cu B => $_a ; _b$. A intersectat cu B => $_a, _b$.
 $A \setminus B$ => P, not(Q)

% 2. utilizarea lui "not" pt a testa tot.

% "A = B" inseamna "orice element din A e si in B (si invers)",
 % dar inseamna si "NU exista element din A care sa NU fie in B (si invers)"
 % not-ul din prolog e practic o cautare de element care sa nu satisfaca.

```
% EXEMPLU (  $A \cup (B \wedge C) = (A \cup B) \wedge (A \cup C)$ ), notand  $\wedge$  = intersectie
membruldr(A,B,C) :- (A;B) , (A;C).
distrib(A,B,C) :- echiv((A ; B,C), membruldr(A,B,C)).
demdistrib :- not((member(A,[false,true]), member(B,[false,true]), member(C,[false,true]),
write((A,B,C))), nl, not(distrib(A,B,C)))).
% tradus prin: nu exista (elemente din) multimi care sa nu fie distributive (deci
distributivitatea este adevarata). Afisam si la fiecare pas.
```

```
% LAB 3 ne ajuta la demonstratii (listaValBool(L))! Exemplu: Sa demonstram ca:
A\B=A\((A^B)).
difinters(A,B) :- echiv((A,not(B)), (A,not((A,B)))).
demdifinters :- not((listaValBool([A,B]), not(difinters(A,B)))).
```

Laboratorul 3

Contine: operatii pe multimi, relatii, functii

`:- [lab3lmc4].` % ASA INCARCAM LABORATORUL ca sa mearga predicatele:

`inversa(L,Reverse_list).`

`afislista(L).`

`sterge(X,L,Res).` % sterge elementul X din lista L.

`stergeuna(X,L,Res).` % sterge prima aparitie. Esueaza daca nu exista apartii

`elimdup(L_initial,L_final).`

`prodmult(M1,M2,M1xM2).` % produs cartezian de multimi.

`prodlist(L1,L2,L1xL2).` % produs cartezian de liste (adica si cu duplicate).

listaBool(L) % L incepe cu N variabile. Acestea vor primi 2^N valori de adevar (true/false).

`listaValBool(L)` % le si afiseaza. `listaVal(Variabile, ValoriPosibile).` % forma mai generala

`append(A,B,C)` # e in prologul normal. C este A urmat de B.

`reuniune(A,B,AUB).`

`intersectie(A,B,I).`

`dif(A,B, A_minus_B).`

`difsim(A,B,A_triunghi_B)` # diferenta simetrica, adica $(A \setminus B) \cup (B \setminus A)$

`egalmult(A,B)` % in Prolog conteaza ordinea elementelor. E definit ca `:- permutare(A,B).`

`permutare(P1,P2)` % intern, prin `stergeuna` verifica pe rand.

`permutarile(L,LP).` % in LP va fi lista de permutari pt L

`sublista(SubLista, Lista)` % prin SubLista putem genera toate sublistele.

`sublistele(L,LS)` % LS returneaza lista sublistelor lui L.

`submultime(S,L) :- permutare(L,P), sublista(S,P).` % problema e ca genereaza mai multe solutii (true-uri) daca nu vrei sa le generezi.

`relbin(R,A,B)` # R sa fie o relatie binara de la A la B.

`relatiibinare(A,B,LR).` % rezultatul ca o lista.

`functie(Functie, A, B).` % Functie va genera functiile de la A la B. `functiile(A,B,LF)` da lista.

`functional(Functionala, A).` %sa nu existe 2 valori pt $f(x)$ (cum se putea intampla la o relatie)

% `fctpart(R,A,B) :- relbin(R,A,B), functional(R,A).`

% nota: functia e functionala + totala (are valoare pt toate elementele din domeniu)

totala(R,A) % verifica daca relatia R are toate elementele din A duse undeva.

`reltotala(R,A,B).` % genereaza toate relatiile totale.

invrel(R,R_inv) % practic dam swap la toate tuplurile => relatie de la B la A.

`injectiv(R).` `relinjectiv(R,A,B).` % ghici ce fac.

surjectiv(R,B). `relsurjectiv(R,A,B)` % ghici ce fac.

`functiebijectiv(F,A,B)` % genereaza functiile bijective

`faradupl(Relatie).` % ghici ciuperca ce-i.

Laboratorul 4

Contine: mai nimic

`:- [lab4lmc1]. % ASA INCARCAM LABORATORUL ca sa mearga predicatele
[s-au rescris altfel functiile din lab precedent... variantele vechi vor fi suprascrise]`

`prodNmult(Lista_Multimi, Rezultat) % produsul cartezian a N multimi.
putereaNmult(A, N, A_la_puterea_N). % putem: putereaNmult([false,true],2,PerCtBool)
im(R,A,Im) % imaginea pe relatia R.
preim(R,B,PreIm).`

`/* definitia imaginii: im(_,_,[]). % cazul de baza, care opreste recursia.
im(R,A,Im) :- setof(Y, X^(member(X,A), member((X,Y),R)), Im), !.
% Gasim toti Y-cii pentru care exista (simbolul ^ in prolog), X din A. */`

[200 linii de predicate mai fara sens.]

Laboratorul 5

Contine: relatii, ordine.

`:- [lab5lmc4].` % ASA INCARCAM LABORATORUL ca sa mearga predicatele

`comprel(S,R,SoR).` % compurerea de relatii binare

`prodrel(R,Q,RxQ).` % produs (practic cartezian) de 2 relatii binare

`diagonala(A, Diag).` % ia fiecare element din A si il pune in relatie cu el insusi.

`putere(R,N,RlaN)` % sau cu 4 parametrii: `putere(_,A,0,Rez) :- diag(A,Rez)` pentru a avea R^0

Proprietati relatii:

`reflexiva(R,A)` % adevarat daca pt orice X din A, exista (X,X) in R

`simetrica(R)` % adevarat daca pt orice (X,Y) din R, exista si (Y,X) in R.

`tranz(R)` % adevarat daca pt orice (X,Y), (Y,Z) din R, atunci si (X,Z) in R.

`preord(R,A) :- refl(R,A), tranz(R).` % preordinea

`eq(R,A) :- preord(R,A), sim(R).` % echivalenta

`antisim(R)` % at % Antisimetria, pt orice (X,Y) din R, (Y,X) nu trebuie sa existe in R.

`asim(R)` % ca " $\leq$ ". Asimetria ermite (X,X) sa existe.

`irefl(R)` % nu exista (X,X) in relatie

`ordine(R,A) :- preord(R,A), antisimetrica(R).` % `relordine(R,A) :- relbinara(R,A), ordine(R,A).`

`ordstricta(R) :- irefl(R), tranz(R).` % ordine stricta.

`(relatieeq(A, R) :- setof(R, releq(R,A), EqA).` genereaza toate relatiile de echiv pe A)

`clasa(X,E,C)` % C = toate elementele cu care X din E este echivalent

`parteq(E,A,P)` % relatia echivalenta E pe multimea A $\Rightarrow$ P = partiia asociata relatiei (clasele)

`partitii(A,PartA)` % ia toate partiile posibile pe multimea A (pt 3 elemente sunt 5 partitii)

`eqpart([A|LA],E)` % Relatia de echivalenta asociata unei partitii

`ordstrdinord(O,S)` % Extrage din ordinea O, ordinea stricta S.

`succdinord(O,Succ)` % data ordinea O, cream Succesorul.(scoatem reflexiv si tranz)

`inchrefl(Rel_initala,A,Rel_reflexiva)` % inchiderea reflexiva (reunim cu diagonala)

`inchsim(Rel,S)`

`inchidtranz(Rel,T)`

`preordgen(Q,A,P) :- inchidtranz(Q,T), inchrefl(T,A,P).` % preordinea pe relatia Q a multimii A.

`eqgen(Q,A,E) :- inchsim(Q,S), preordgen(S,A,E).` % echivalenta generata

`orddinsucc(Succ,A,O) :- preordgen(Succ,A,O).` % construim un poset (partially ordered set) din ordinea de succesiune Succ. (inchidem reflexiv si tranzitiv)

Laboratorul 6

Contine: Poseturi, Latici

:- [lab6lmc3]. % ASA INCARCAM LABORATORUL ca sa mearga predicatele

Exemplu construire poset:

```
romb(R,OrdR) :- R=[0,a,b,1],
orddinsucc([(0,a),(0,b),(a,1),(b,1)],R,OrdR).
```

Latici comune:

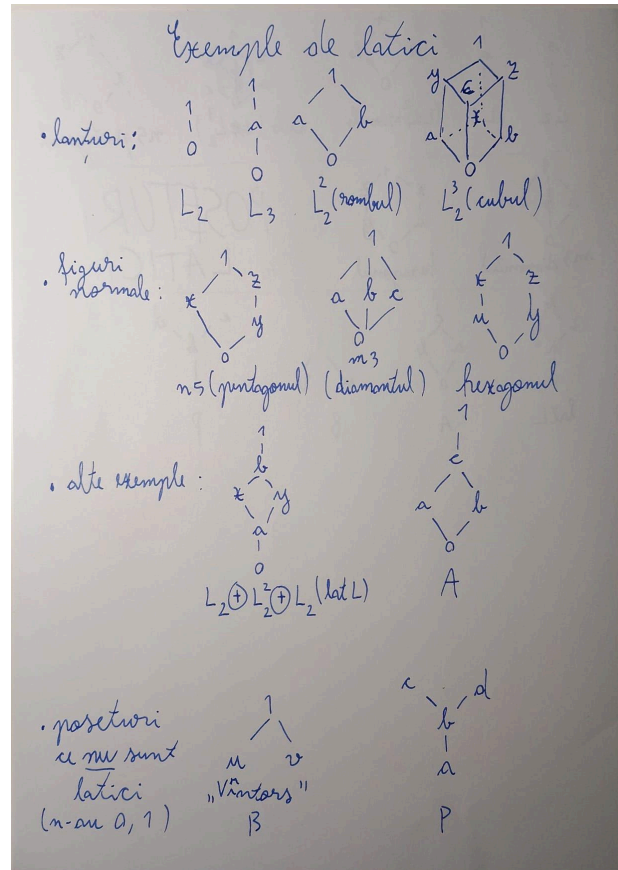
```
l2([0,1],OrdL2) % Lantul L2
l3(L3,OrdL3) % Lantul L3
romb(R,OrdR) % Romb = L2^2
cub(C,OrdC) % Cub = L2^3
m3(M3,OrdM3) % Diamantul = M3 (3 in mij)
n5(N5,OrdN5) % Pentagonul: N5
hexa(M,Ord) % Hexagonul
```

```
posetA(A,OrdA) % OrdA = relatia L2^2 + L2
```

```
latL(L,OrdL) % L2+L2^2+L2
```

```
posetB(B,OrdB) % OrdB = "V rasturnat"
```

```
posetP(P,OrdP) % OrdP = L2 + "V"
```



pastrel(F,R,S) % daca functia F e "parte stabila" = duce orice elemente din relatia R in rel S
% Functiile crescatoare F:P->Q intre doua poseturi (P,OrdP) si (Q,OrdQ):

fctcresc(F,P,OrdP,Q,OrdQ) :- functie(F,P,Q), pastrel(F,OrdP,OrdQ).

fctstrcresc(F,P,OrdP,Q,OrdQ).

% Pe exemplele de mai sus, model de rezolvare:

% functii crescatoare injective de la posetA la posetB

fctlecrestinjAlaB(LF) :- posetA(A,OrdA), posetB(B,OrdB),

setof(F, (fctcresc(F,A,OrdA,B,OrdB), injectiv(F)), LF), !.

fctlecrestinjAlaB([]). % daca nu exista va returna multimea vida.

% interogare daca fct crescatoare de la B la L2. spoiler: (0,0,0),(0,0,1),(1,0,1),(1,1,1).

?- posetB(B,OrdB), l2(L2,OrdL2), fctcresc(F,B,OrdB,L2,OrdL2).

Minoranti si p-acolo

```
% M minorant, S submultime, (P,OrdP) posetul.
minoreaza(M,S,OrdP) :- not((member(X,S), not(member((M,X),Ord)))).
minorant(M,S,P,OrdP) :- member(M,P), minoreaza(M,S,OrdP).
minorantii(S,P,OrdP,LM) :- setof(M, minorant(M,S,P,OrdP), LM), !.
min(S,OrdP,M) :- minorant(M,S,S,Ord).
inf(S,P,OrdP,M) :- minorantii(S,P,OrdP,LM), max(LM,OrdP,M).
elemminimal(M,S,Ord) :- ordstrdinord(Ord,OrdStr), nueminstrict(M,S,OrdStr). # pt V sunt 2
elemente maxime. Exista si elementeleminimale
```

Similar pentru: majoreaza, majorant, majorantii, max, sup, elemmaximal

Latici

```
latice(L,OrdL) :- not((member(X,L), member(Y,L),
    not((inf([X,Y],L,OrdL,_), sup([X,Y],L,OrdL,_)))). % toate elem au inf si sup
latmarg(L,OrdL) :- latice(L,OrdL), min(L,OrdL,_), max(L,OrdL,_).
complem(X,L,OrdL,Y) :- min(L,OrdL,Zero), max(L,OrdL,Unu), member(Y,L),
    inf([X,Y],L,OrdL,Zero), sup([X,Y],L,OrdL,Unu). % determina Y complementul
complem(X,L,OrdL,Zero,Unu,Y) % complement daca stim 0 si 1.
inchcomplem(S,L,OrdL). % S va fi inchiderea.
laticemargcomplem(L,OrdL) :- latmarg(L,OrdL),
    not((member(X,L), not(complem(X,L,OrdL,_)))).
lant(P,OrdP) :- not((member(X,P), member(Y,P), % sa nu existe 2 elemente necomparabile
    not((member((X,Y),OrdP) ; member((Y,X),OrdP))))).
sublant(S,L,OrdL) :- sublista(S,L), lant(S,OrdL). % si setof(S, sublant(S,L,Ord), LS) pt toate

sublat(S,L,OrdL) :- sublista(S,L), not((member(X,S), member(Y,S),
    inf([X,Y],L,OrdL,XsiY), sup([X,Y],L,OrdL,XsauY),
    not((member(XsiY,S), member(XsauY,S))))).
sublatmarg(S,L,OrdL) :- sublat(S,L,OrdL), inch0si1(S,L,OrdL).
sublatnelinord(S,L,OrdL) :- sublat(S,L,OrdL), not(lant(S,OrdL)). %sublatici nelinier ordonate
toatelantsublat(L,OrdL) :- not((sublant(S,L,OrdL), not(sublat(S,L,OrdL)))). % verifica daca
toate sublanturile (poseturi linier ordonate) sunt sublatici.

% ex utilizare: Sublaticile si sublaticile marginite ale lui  $L_2+L_2^2+L_2$ :
% sublaticileL(LS) :- latL(L,OrdL), setof(S, sublat(S,L,OrdL), LS), !. sublaticileL([]).
% sublaticilemargL(LS) :- latL(L,OrdL), setof(S, sublatmarg(S,L,OrdL), LS), !.
sublaticilemargL([]). % rezultatele vor fi 52 elemente, respectiv 13 (cele care au si 0 si 1)
```

Proprietati latice si algebre boole

```
subalgbbool(S,B,OrdB) :- sublatmarg(S,B,OrdB), inchcomplem(S,B,OrdB).
%ex:subalgebre cub subalgbboolCub(LS):-cub(B,OrdB), setof(S, subalgbbool(S,B,OrdB), LS),!.
```

```
morflat(F,L,OrdL,M,OrdM) :- functie(F,L,M), pastrdisjconj(F,L,OrdL,M,OrdM).
pastrdisjconj(F,L,OrdL,M,OrdM) :- not((member(X,L), member(Y,L),
    member((X,FX),F), member((Y,FY),F), %  $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$ 
    inf([X,Y],L,OrdL,XsiY), sup([X,Y],L,OrdL,XsauY),
    inf([FX,FY],M,OrdM,FXsiFY), sup([FX,FY],M,OrdM,FXsauFY),
    not((member((XsiY,FXsiFY),F), member((XsauY,FXsauFY),F))))).
morflatmarg(F,L,OrdL,M,OrdM) :- morflat(F,L,OrdL,M,OrdM), pastr0si1(F,L,OrdL,M,OrdM).
pastrcomplem(F,L,OrdL,M,OrdM)
morfbbool(F,L,OrdL,M,OrdM) :- morflatmarg(F,L,OrdL,M,OrdM),
    pastrcomplem(F,L,OrdL,M,OrdM).
```

%exemple pt alte concepte:

```
morfBooleCublaRomb(LF) :- cub(C,OrdC), romb(R,OrdR),
    setof(F, morflatmarg(F,C,OrdC,R,OrdR), LF), !.
morfBooleCublaRomb([]).
```

Automorfisme booleene cub.

```
automBooleCub(LF) :- cub(C,OrdC),
    setof(F, (morflatmarg(F,C,OrdC,C,OrdC), injectiv(F), surjectiv(F,C)), LF), !.
automBooleCub([]). % Varianta lenta (dar simpla)

automorfbooleeneCub(LF) :- cub(C,OrdC), Cminus01=[a,b,c,x,y,z],
    setof(F, (P,G)^(permutare(Cminus01,P), fctbij(G,Cminus01,P),
    append([(0,0),(1,1)],G,F), pastrdisjconj(F,C,OrdC,C,OrdC)), LF), !.
automorfbooleeneCub([]). % deci, exista functia bijectiva G (a unei permutari P ce exista
    pt laticea initiala) la care readaug 0,1 si si are disjunctia fata de conjunctie.
```

/* Sa de determine toate functiile de la $L_2+L_2^2$ la L_3 care pastreaza compunerea relatiei de succesiune cu ea insasi, si sa se arate ca niciuna nu e injectiva. */

```
l2plusromb(L,OrdL) :- L=[0,a,b,c,1],
    orddinsucc([(0,a),(a,b),(a,c),(b,1),(c,1)],L,OrdL). % hardcodam posetul dat

fctposetarb(P,OrdP,Q,OrdQ,LF) :- succdinord(OrdP,SuccP), % scoatem refl si tranz
    succdinord(OrdQ,SuccQ), % compunem ca in cerinta succesiune cu ea
    comprel(SuccP,SuccP,SuccPoSuccP), comprel(SuccQ,SuccQ,SuccQoSuccQ),
    setof(F, (functie(F,P,Q), pastrrel(F,SuccPoSuccP,SuccQoSuccQ)), LF), !.
fctposetarb(_,_,_,_). % setof gaseste toate functiile care pastreaza relatiile.

fctcerute(LF) :- l2plusromb(L,OrdL), l3(L3,OrdL3),
    fctposetarb(L,OrdL,L3,OrdL3,LF).
nusuntinj(LF) :- not((member(F,LF), write(F), nl, injectiv(F))).
testneinjerut :- fctcerute(LF), nusuntinj(LF).
```


Laboratorul 7

Contine: mai multe demonstratii semantice

`:- [lab7lmc2]. % ASA INCARCAM LABORATORUL ca sa mearga predicatele`

```
/* Sa demonstram ca, pentru orice multimi de enunturi Sigma1, Sigma2, Sigma3 si orice
enunturi fi, psi, hi, are loc regula de deductie:
```

```
    Sigma1 U {fi} |- psi, Sigma2 U {psi^hi} |- fi, Sigma3 U {psi} |- hi
```

```
    -----
    Sigma1 U Sigma2 U Sigma3 |- fi <-> (psi^hi)
```

```
*/
```

```
ipoteza1(Fi,Psi) :- implica(Fi,Psi).
```

```
ipoteza2(Fi,Psi,Hi) :- implica((Psi,Hi),Fi).
```

```
ipoteza3(Psi,Hi) :- implica(Psi,Hi).
```

```
concluzia(Fi,Psi,Hi) :- echiv(Fi,(Psi,Hi)).
```

```
demregded :- not((listaValBool([Fi,Psi,Hi]), ipoteza1(Fi,Psi),
    ipoteza2(Fi,Psi,Hi), ipoteza3(Psi,Hi), not(concluzia(Fi,Psi,Hi))))).
```

```
/* Fie alfa, beta, gama, delta, epsilon in V.
```

```
Sa demonstram ca multimea:
```

```
M = {alfa -> beta, beta -> (gama^delta), gama -> alfa, delta -> -|alfa}
```

```
e satisfiabila. */
```

```
enuntul1(A,B) :- implica(A,B).
```

```
enuntul2(B,C,D) :- implica(B,(C,D)).
```

```
enuntul3(A,C) :- implica(C,A).
```

```
enuntul4(A,D) :- implica(D,not(A)).
```

```
satisfiabila :- listaBool([A,B,C,D,E]), enuntul1(A,B), enuntul2(B,C,D),
    enuntul3(A,C), enuntul4(A,D), write((A,B,C,D,E)), nl. % se opreste la prima sol.
```

```
/* Fie signatura tau = (1;2;0) si algebra (A,f,R,k) de aceasta signatura, cu multimea suport
A = {a,b,c}, avand |A|=3 (i.e. a/=b/=c/=a), R relatie de ordine pe A a.i. c = min(A,R),
iar a,b sunt elementele maxime ale posetului (A,R), f : A->A strict crescatoare de la
posetul (A,R) la dualul sau (A,R^-1) cu f(c)=a, iar k=max(A,R^-1). Sa se determine daca:
```

```
A |= \-/x \-/y [f(x)=y -> (R(x,y) v y=k)]. */
```

```
multA([a,b,c]). % multimea A data in enunt
```

```
ordA(R) :- multA(A), relord(R,A), min(A,R,c), elemmax(a,A,R), elemmax(b,A,R).
```

```
ctK(K) :- multA(A), ordA(R), invrel(R,Q), max(A,Q,K). %constanta k = c (inversul lui "V")
```

```
fctF(F) :- multA(A), ordA(R), invrel(R,Q), % aducem multimea, relatia si inversa relatiei.
```

```
    ordstrdinord(R,S), ordstrdinord(Q,T), % luam ordinea stricta (f strict cresc)
```

```
    functie(F,A,A), pastrel(F,S,T), member((c,a),F). %prima functie ce pastreaza relatia
```

```
verifAsatenunt :- multA(A), ordA(R), fctF(F), ctK(K), % luam ipotezele.
```

```
    not((member(X,A), member(Y,A), write((X,Y)), nl, % = nu exista X,Y din A
```

```
    not(implica(member((X,Y),F), member((X,Y),R) ; Y=K))))). % pt care nu se satisf.
```

Examene din trecut

februarie septembrie 2023 (lista 1):

Exercitiul 1. Sa se determine toate functiile strict crescatoare $f : L_4 \rightarrow L_2^3$ si sa se arate ca toate aceste functii sunt morfisme de latici marginite. PROLOG: predicat unar + zeroar. SOLUTIE

:- [lab7lmc2]. % includem tot sa fim siguri :)

% nu aveam lantul L_4 predefinit. el este $0 \leq a \leq b \leq 1$.
 $l4(A, Ord) :- A = [0, 1, a, b], \text{orddinsucc}([(0, a), (a, b), (b, 1)], A, Ord).$

% luam cele 2 poseturi din enunt. adaugam toate functiile F strict crescatoare in lista
% de functii ceruta, FctCresc:
 $\text{fctL4laL2xL2xL2}(F\text{ctCresc}) :- l4(L_4, \text{OrdL}_4), \text{cub}(\text{Cub}, \text{OrdCub}),$
 $\text{setof}(F, \text{fctstrcresc}(F, L_4, \text{OrdL}_4, \text{Cub}, \text{OrdCub}), F\text{ctCresc}).$

% luam cele 2 poseturi din enunt. generam acele functii strict crescatoare. verificam
% sa nu existe functii care sa nu fie morfisme de latici marginite.
 $\text{toatemorflatmarg} :- l4(L_4, \text{OrdL}_4), \text{cub}(\text{Cub}, \text{OrdCub}),$
 $\text{fctL4laL2xL2xL2}(F\text{ct}),$
 $\text{not}((\text{member}(F, F\text{ct}), \text{not}(\text{morflatmarg}(F, L_4, \text{OrdL}_4, \text{Cub}, \text{OrdCub}))))).$

Exercitiul 2. Fie V multimea variabilelor propozitionale si E multimea enunturilor logicii propozitionale clasice, iar α, β, fi in E astfel incat: $\text{fi} = (\alpha \rightarrow \beta) \rightarrow (\alpha \wedge \beta)$

Sa se demonstreze ca, in logica propozitionala clasica, cu demonstratie semantica:

-propr1. Daca α si β in V , atunci fi e satisfiabil

-propr2 Daca fi e teorema formală, multimea $\{\alpha, \beta\}$ e nesatisfiabila

$\text{fi}(A, B) :- \text{implica}(\text{implica}(A, B), (A, \text{not}(B)))$.
 $\text{propr1} :- \text{listaBool}([Alfa, Beta]), \text{fi}(Alfa, Beta).$

% nota: a satisface o multime inseamna a satisface toate elementele.
 $\text{propr2} :- \text{not}((\text{listaBool}([Alfa, Beta]), \text{fi}(Alfa, Beta), (Alfa, Beta))))).$

Exercițiul 3. Considerăm semnătura de ordinul I: $T = (1, 2, 0)$, simbolul de operație unară f , simbolul de relație binară R și simbolul de constantă k , o mulțime $A = \{a, b, c, d\}$ având $|A| = 4$

$f: A \rightarrow A$ este relația funcțională totală a cărei închidere reflexivă coincide cu închiderea reflexivă a relației de succesiune a mulțimii total ordonate $(A, \leq)$ având $\min(A, \leq) = a$, $\max(A, \leq) = d$ și $b < c$ (unde $<$ este relația de ordine strictă asociată relației de ordine $\leq$);

R este relația de ordine strictă a posetului $(A, \subseteq)$ izomorf cu romb L_2^2 având $\min(A, \subseteq) = a$ și $\max(A, \subseteq) = d$;

$k \in \{b, c, d\}$ și are proprietatea că există $p \in A$ astfel încât (k, p) aparține f inters R .

Considerăm două variabile distincte $x, y \in \text{Var}$ și enunțul:
 $e = \forall x \forall y (f(x) = k \wedge f(k) = y \wedge R(k, y) \rightarrow R(x, y))$.

Se determină funcția f , relația binară R și constanta k , apoi să se determine dacă A satisface e .

$\text{multimeaA}([a, b, c, d])$.

$\text{detf}(F) :- F = [(a, b), (b, c), (c, d), (d, d)], \text{multimeaA}(A), \text{functională}(F, A), \text{totală}(F, A)$.
 $\text{detR}(R) :- \text{multimeaA}(A), \text{orddinsucc}([(a, b), (a, c), (b, d), (c, d)], A, \text{Ord}), \text{ordstrdinord}(\text{Ord}, R)$.

$\text{intersecțieRelatii}(R1, R2, \text{Inters}) :- \text{setof}((X, Y), (\text{member}((X, Y), R1), \text{member}((X, Y), R2)), \text{Inters})$.

$\text{intersecție_F_cu_R}(\text{Inters}) :- \text{detR}(R), \text{detf}(F), \text{intersecțieRelatii}(F, R, \text{Inters})$.

$\text{detK}(K) :- \text{member}(K, [b, c, d]), \text{intersecție_F_cu_R}(\text{Inters}), \text{member}((K, _), \text{Inters})$.

$\text{verifAsatεpsilon} :- \text{multimeaA}(A), \text{detR}(R), \text{detK}(K), \text{detf}(F),$
 $\text{not}((\text{member}(X, A), \text{member}(Y, A),$
 $\text{not}(\text{implica}(\text{member}((X, K), F), \text{member}((K, Y), F), \text{member}((K, Y), R)), \text{member}((X, Y), R))))$.